Server



MHS: Onward Towards Industry Adoption - Demonstrating Modular Plug-and-Play (M-PNP) Tools and POC Work

Munir Ahmad *Lattice Semiconductor, Chief Compute Architect*

Phil Leech *HPE, Principal Architect / Technologist and OCP MHS M-PNP Workstream Lead*

Bill Scherer *HPE, Principal Architect / Technologist*

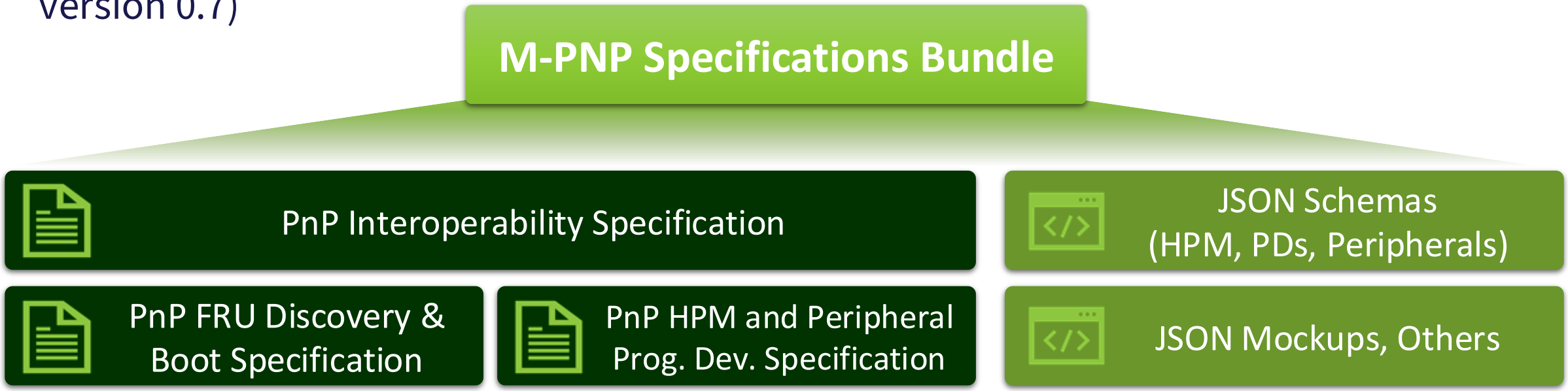
HPE Lattice

Agenda

- M-PNP Workstream and Specifications Overview
- Component JSON Self Describing Files Usage
- M-PNP Tools To Assist Industry Adoption
- Prototyping and Proof of Concept Work
- Schedule & Call to Action

What is the DC-MHS M-PNP Workstream?

- MHS workstream to standardize component interoperability
- Helps ease burden of system firmware, BIOS and platform enablement integration of new HPMs and Peripherals
- Drive component conformance for system integration and validation
- Specs, collaterals, tools, etc. delivered as the M-PNP Specifications Bundle (currently at version 0.7)



M-PNP Specifications Bundle

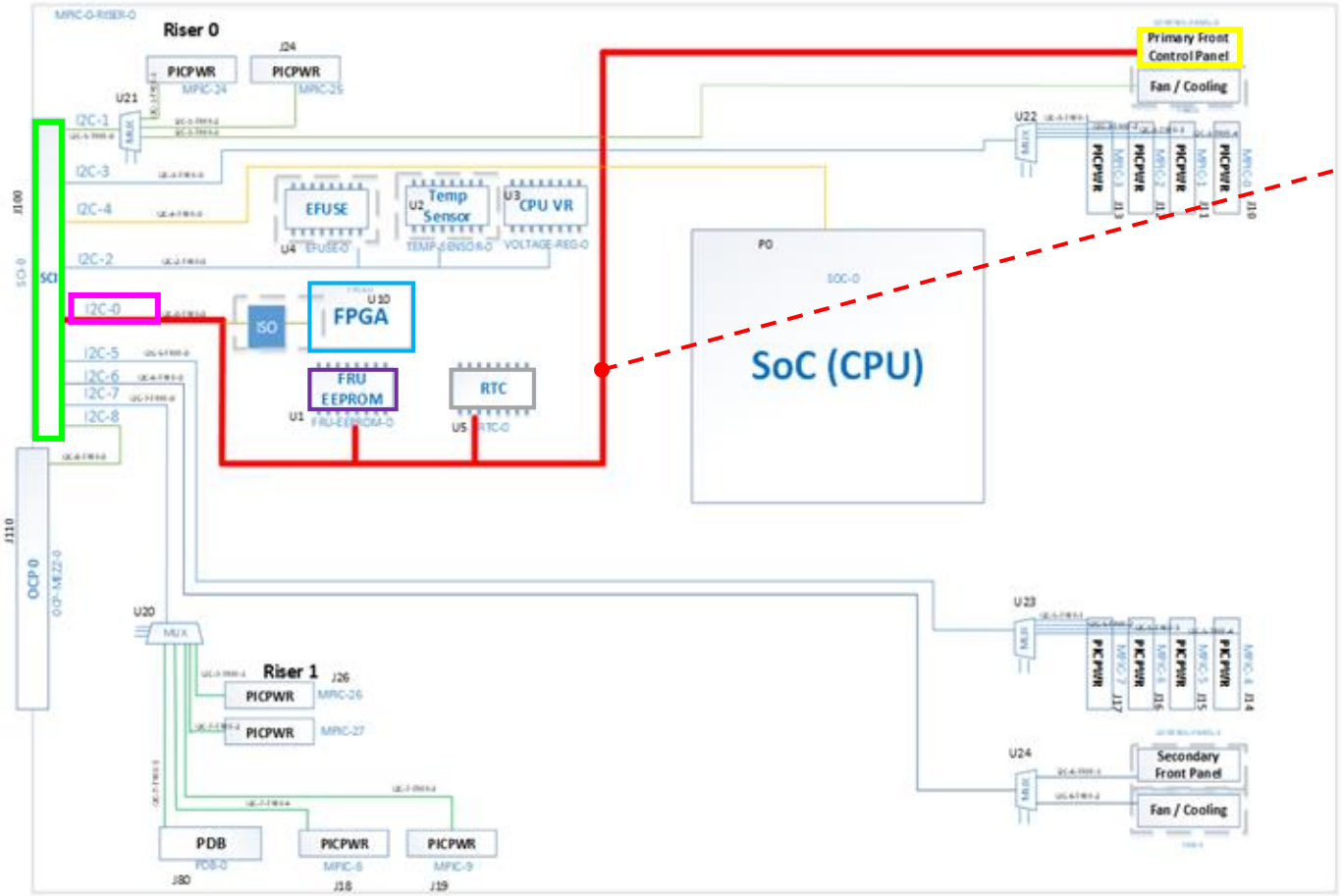
M-PNP Specifications Bundle

PnP FRU Discovery & Boot Specification	Platform discovery process, FRU definition, JSON data and usage, boot requirements, and more.
PnP HPM & Peripherals Programmable Device Specification	Standardized interfaces, provides framework, defines IP blocks (e.g. power management), JSON data and usage, etc.
PnP Interoperability Specification	Provides the scope, vision, common terms, and conformance criteria for interoperability.
JSON Schemas (HPM, PDs, Peripherals)	Contains the HPM / Peripheral and Prog. Dev. JSON schema that defines the required and optional fields for self-describing JSON
JSON Mockups, Others	Set of HPM, Peripheral, and FPGA JSON mock-up files as examples for reference.
Collaterals List - Tools and Reference Code	A manifest of current tools and example code for reference.

Self-describing Hardware

- Self-description is an essential cornerstone for adopting M-PNP
  - Enables dynamic discovery of HPM hardware by the SCM
- We use JSON to capture and organize platform data
  - Extensive library and language support
  - BMCs already have JSON libraries for Redfish – no additional support footprint
- JSON data covers all information needed to boot and manage hardware
  - Hardware buses: presence, parameters, control, MUXes/hubs, connections, bridges
  - Connectors: manufacturer and model, presence, FlexIOs, signals
  - Programmable Devices: memory map, register definitions, signals, virtual wires, buses
  - This is just the highlights from over 5K lines of JSON schema
- Same schema for HPMs and non-endpoint peripherals
  - Simplifies BMC digestion of data

M-PNP JSON – JSON Usage Example



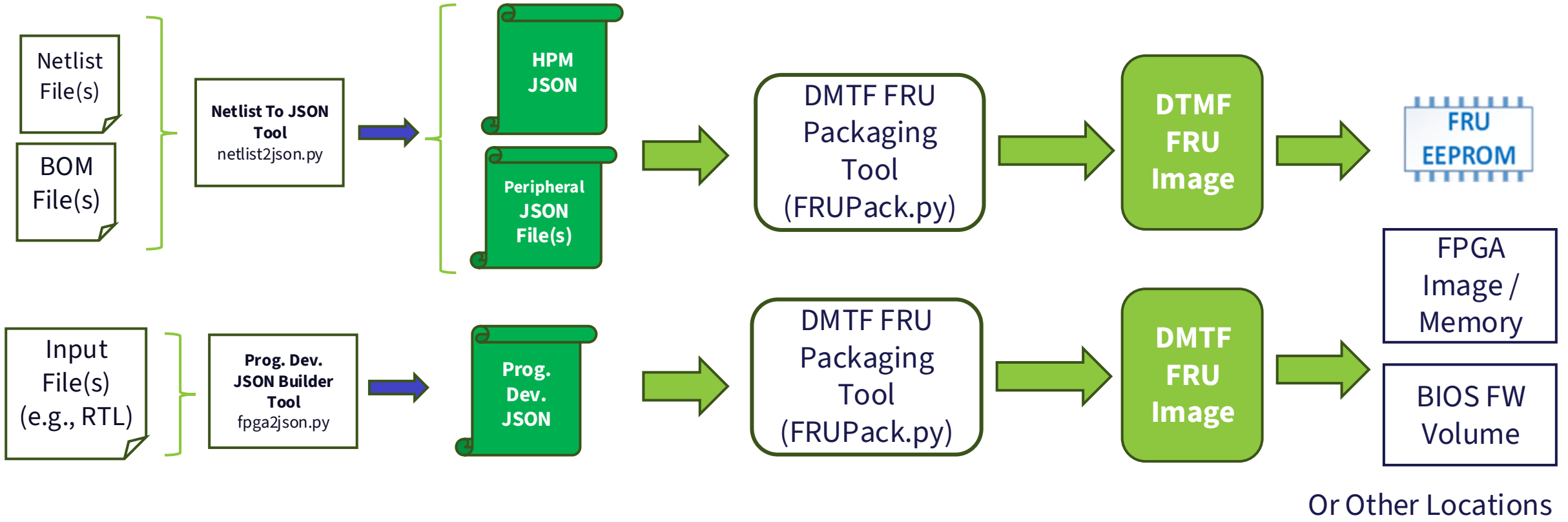

```
"I2C": [
    {
        //contains FRU EEPROM FPGA RTC and Primary control panel connection
        "Identifier": "I2C-0",
        "VoltageTenthsVolts": 33,
        "PowerState": "pre-standby",
        "FrequencyKHz": 400,
        "Description": "FPGA and FRU EEPROM",
        "Trees": [
            {
                "Identifier": "I2C-0-TREE-0",
                "ConnectedDevices": [
                    {
                        "Type": "PassiveNonProgrammable",
                        "Identifier": "FRU-EEPROM-0",
                        "FrequencyMaxKHz": 400,
                        "Address": "0x11"
                    },
                    {
                        "Type": "PassiveNonProgrammable",
                        "Identifier": "RTC-0",
                        "FrequencyMaxKHz": 400,
                        "Address": "0x22"
                    },
                    {
                        "Type": "ActiveProgrammable",
                        "Identifier": "FPGA-0",
                        "FrequencyMaxKHz": 400,
                        "Address": "0x22" ,
                        //if this is true then you can follow MPNP FPGA spec to get to FPGA memory map
                        "RandomAccessCapable":  true
                    }
                ],
                "Connectors": [
                    {
                        "Type": "SCI",
                        "Endpoint": "SCI-0"
                    },
                    {
                        "Type": "ControlPanel",
                        "Endpoint": "CP-0"
                    }
                ]
            }
        ]
    },
```

Getting the JSON to the BMC: FRU

- JSON data describes the hardware
- FRU image provides a standard way for SCM to access it
  - I2C bus 0, address 0xA0 (outside any MUXes)
- Larger size of JSON data requires DMTF FRU format (DSP0220)
  - Adds files that are associated with FRU data
- JSON data file may be *resident* in the FRU EEPROM
  - If not resident, *context* field of FRU file table details location
  - Virtual EEPROM emulated by the FPGA
    - must be built-in to FPGA, not user logic, to be accessible while FPGA is in reset
  - BIOS firmware volume
- Signature in the FRU ensures no tampering with JSON data

JSON Creation and Packaging Flow

Process flow: netlists, BOMs, and RTL -> JSON files -> packaging in DMTF FRU Format -> FRU EEPROM, HPM PD (e.g. FPGA) Image / Memory, BIOS FV, others



M-PNP Tools To Accelerate Adoption

Currently Released

- JSON Validator
- FRU Packaging Tool

Release In Progress

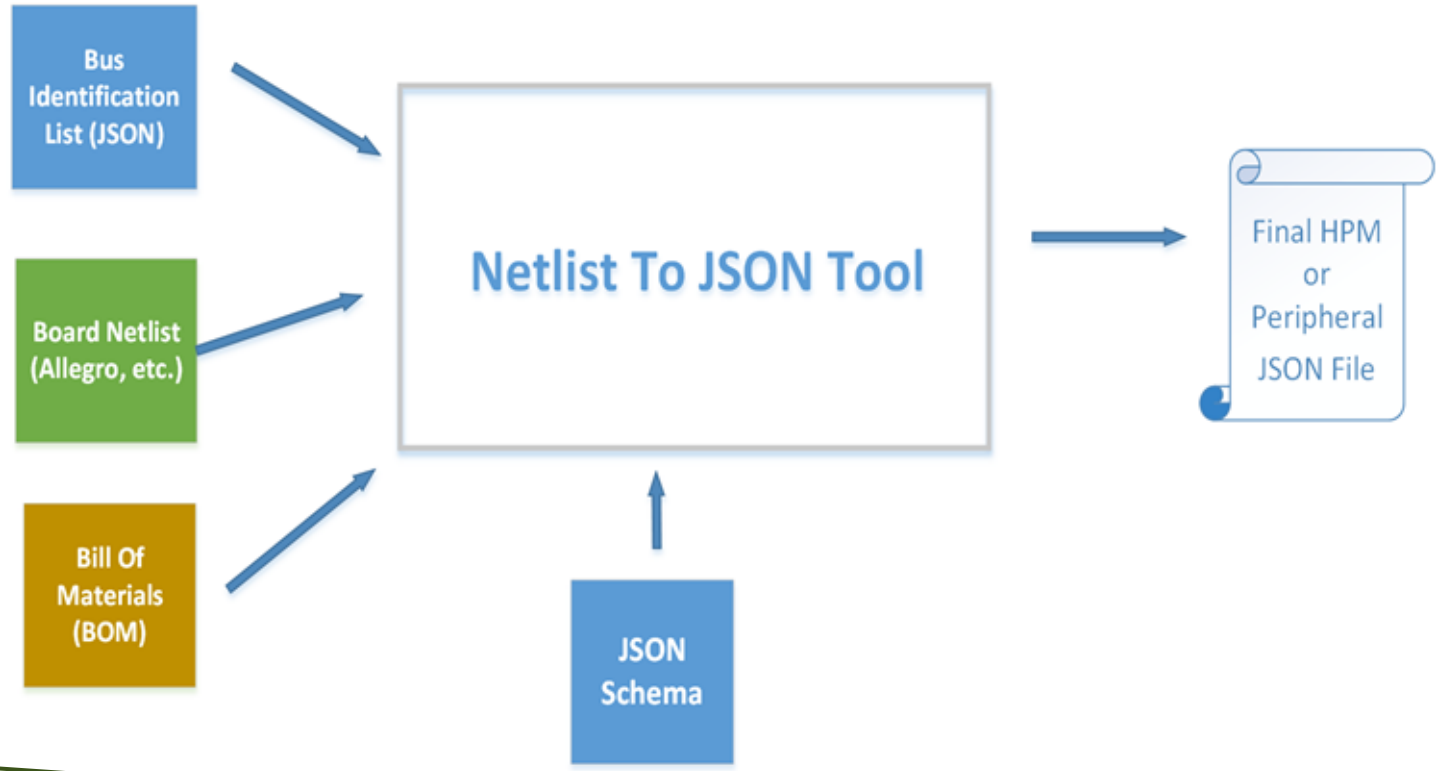
- Netlist to JSON File Creation Tool
- Programmable Device JSON File Creation Tool

Future Development

- JSON To Linux Device Tree Conversion Tool

Netlist To JSON File Creation Tool

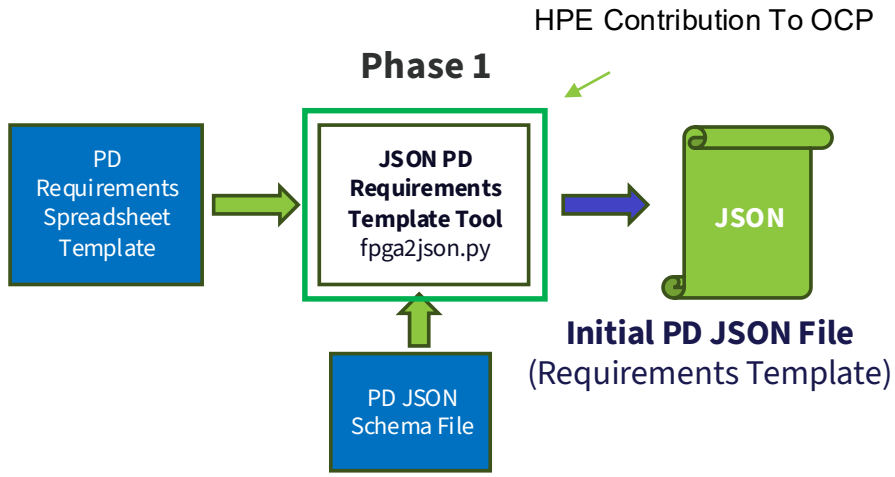
- Workstream members working on JSON file creation tool
  - Convert Cadence Allegro netlist to JSON file
  - Usable for both HPMs and peripherals
  - Requires PCB netlist and BOM files
  - Requires netlist naming convention
- Release timeframe targeted Q4 ‘26

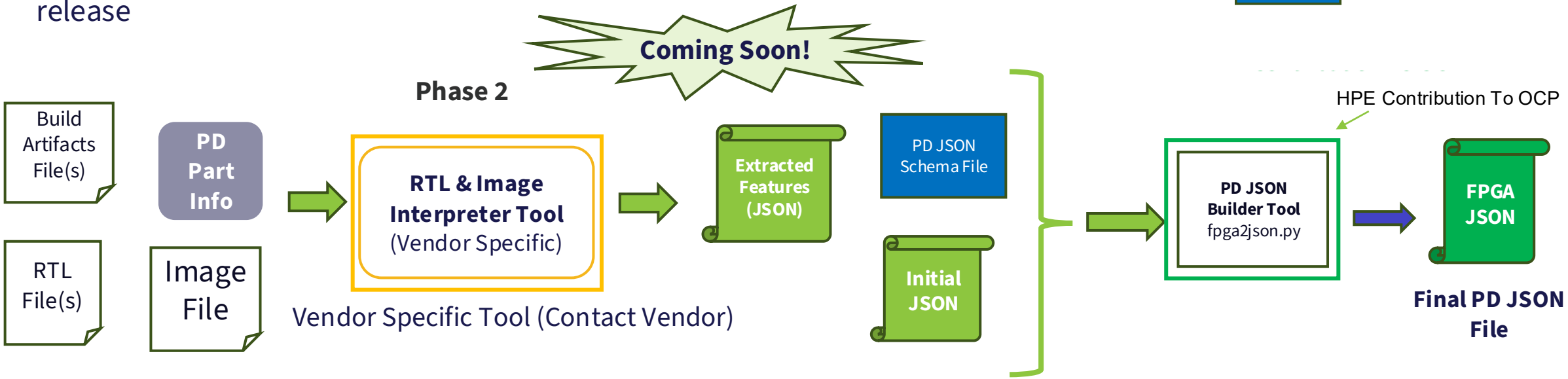


Coming Soon!

Programmable Device JSON File Creation Tool

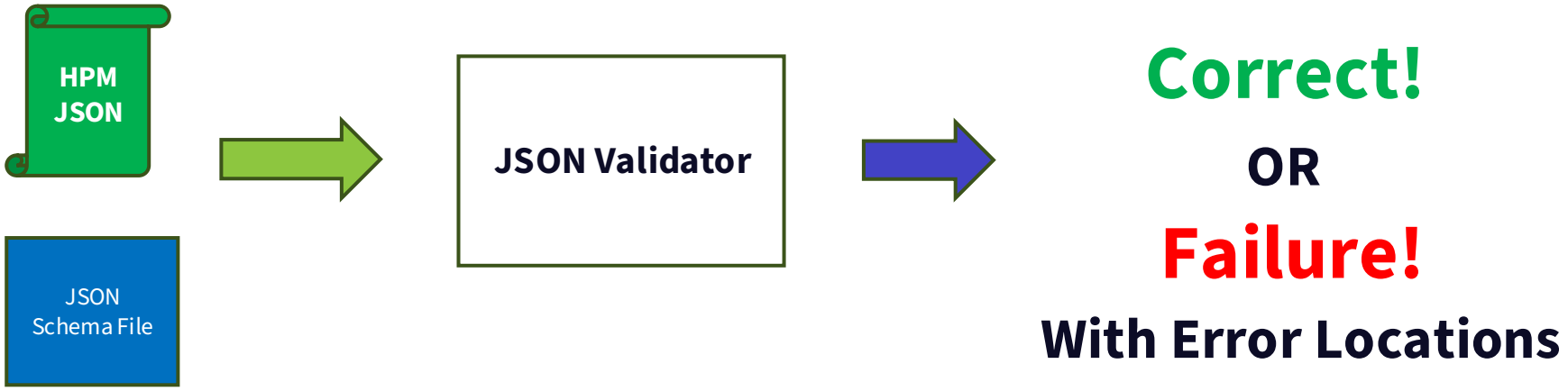
- Programmable Device (PD) (e.g., FPGA) self-describing JSON file automatic creation two phased approach
  - Initial 'requirements' JSON as pre-build input, uses spreadsheet template, schema as inputs
  - Final PD JSON file built from image, RTL, initial JSON, and other build artifacts
- Python script: fpga2json.py (same tool, different function commands)
- Targeting open-source release late Q4 '26 on after the 0.7 spec package release





JSON Validator

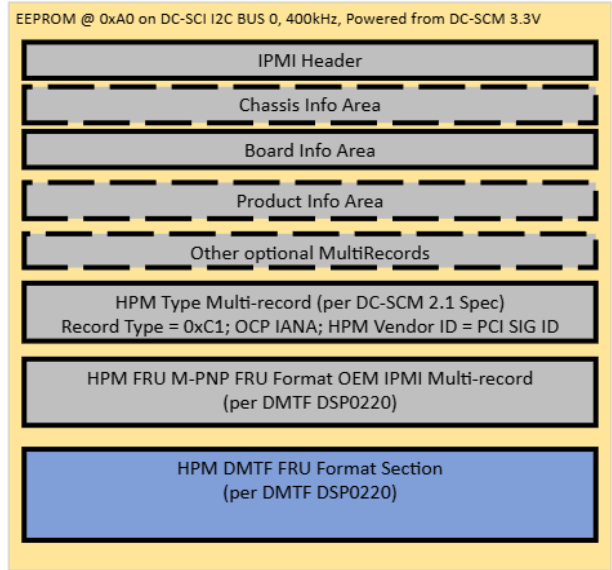
- Validates HPM and Peripheral JSON files for correctness
- Checks JSON against the schema
- Ensures that identifiers and links to different sections correctly match
- Contributed by Intel to OCP GitHub
  - https://github.com/opencomputeproject/OCP-SVR-MHS-M-PNP_JSON_Val_Tool

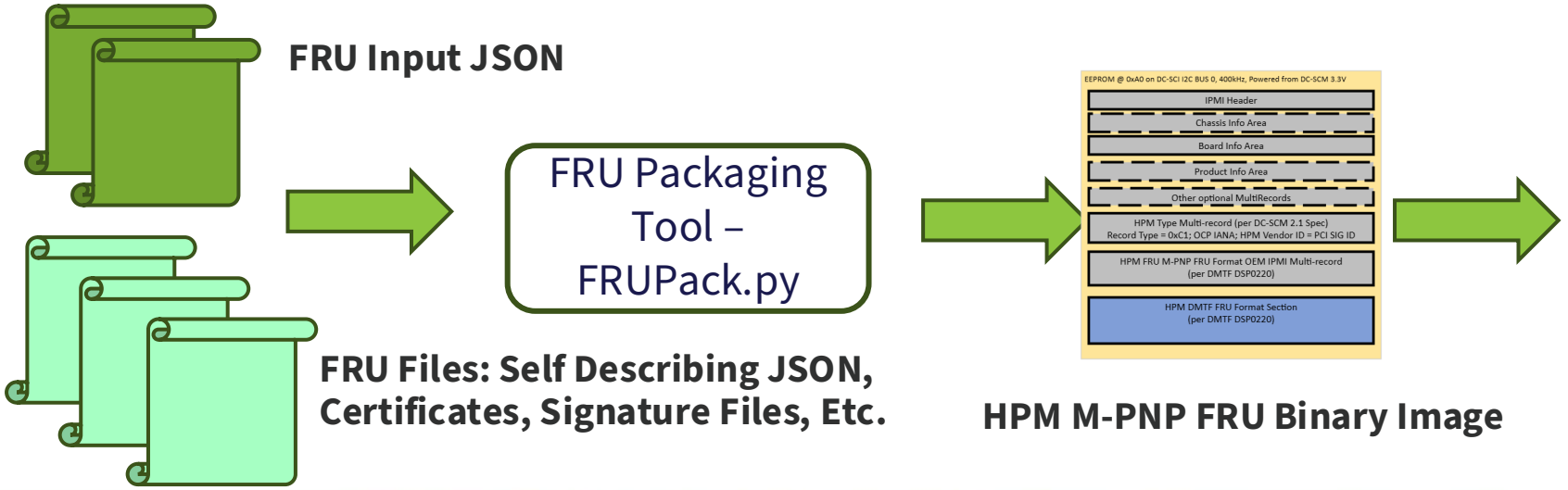


FRU Image Packaging Tool

- Builds IPMI and DMTF FRU files format (DSP0220) combined images
- Python script - FRUPack.py and dependency scripts (utilities, classes, etc.)
- Located on OCP GitHub (https://opencomputeproject/OCP-SVR-MHS-M-PNP_FRUTool · GitHub)

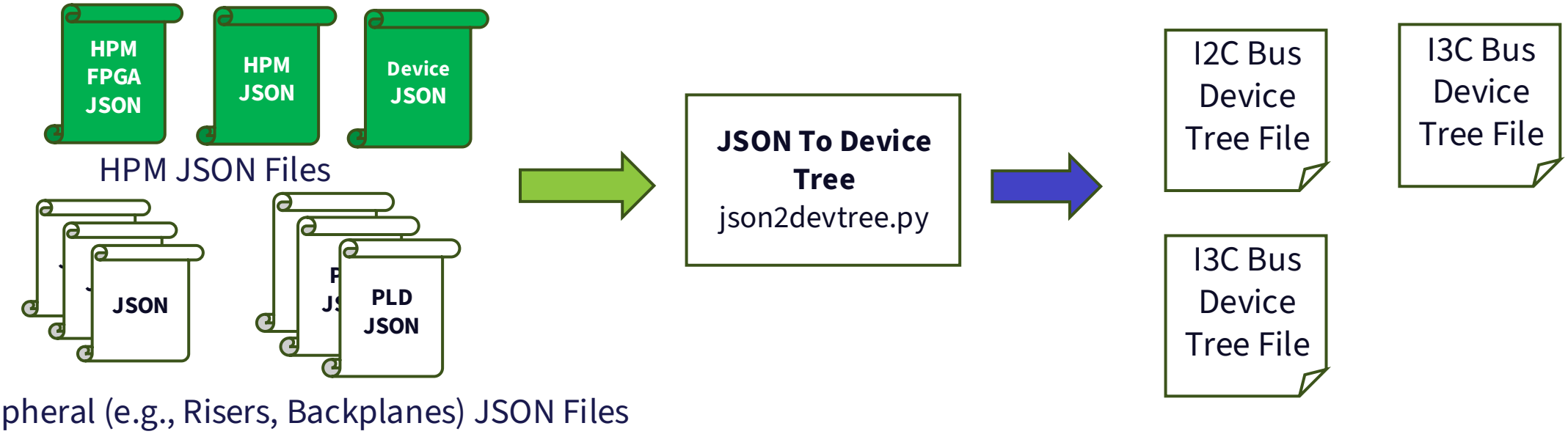
HPM M-PNP FRU Layout





JSON To Linux Device Tree Conversion Tool

- Analyzes HPM, Peripheral, and programmable logic JSON files
- Creates Linux Device Tree files from JSON for various system bus drivers (I2C, I3C, etc.)
- Targeting Q1 ‘27 for initial version



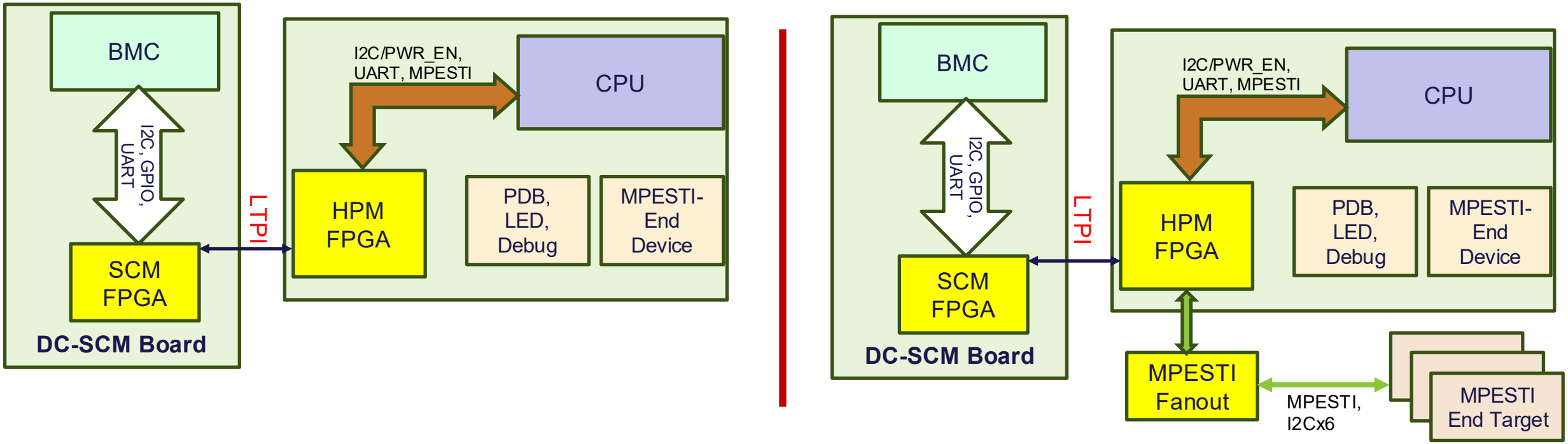
POCs and Contributions Agenda

- HPE Intel Lattice Single Node POC
- Lattice – Intel Dual Node POC
- Reference Contributions

HPE Intel Lattice POC

- DC-MHS plug-and-play solution that standardizes BMC PnP firmware and FPGA capabilities for automatic discovery and integration of HPM platforms via HPM FPGA-JSON

Standardized BMC PnP and FPGA JSON capabilities for automatic discovery and integration	Flexible and Scalable Single and Dual-Node LTPI Connectivity for transferring JSON info to BMC	Use M-PESTI expansion capabilities for end points. Allows fanout and bridging for multi-endpoint connectivity



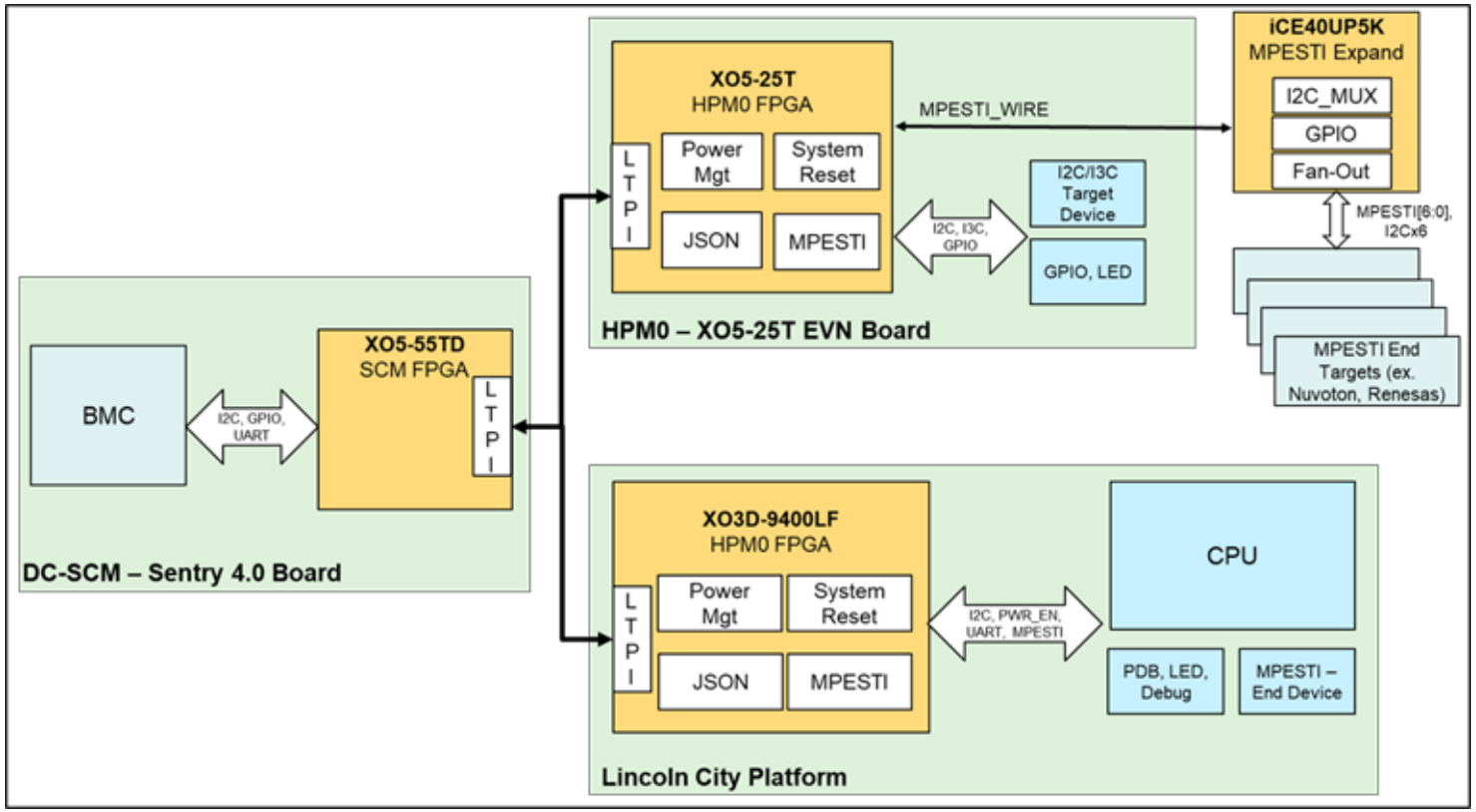
HPE Intel Lattice POC

- This POC presents a DC-MHS plug-and-play solution that standardizes BMC PnP firmware and FPGA capabilities for automatic discovery and integration of HPM platforms via HPM FPGA-JSON. Leveraging LTPI, it enables seamless connectivity to MPESTI, I2C, and I3C devices, reducing integration complexity. Dual-node LTPI support allows independent connection to two HPM platforms or CPUs,

  - **Accelerated Modular Plug-and-Play Integration (M-PNP)**: Standardizes BMC PnP firmware and FPGA capabilities for automatic discovery and integration of HPM platforms via HPM FPGA-JSON.

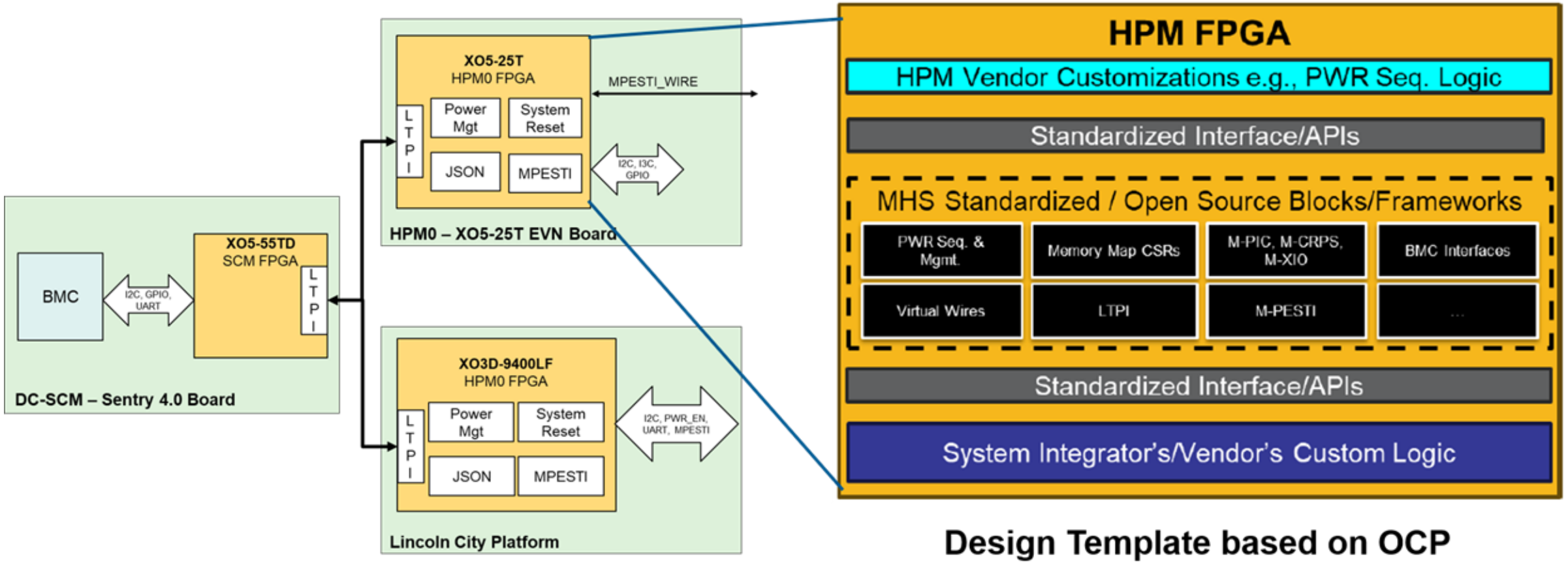
  - **Flexible and Scalable Dual-Node LTPI** Connectivity: Provides an independent, up-to-date dual-node LTPI solution that supports dual HPM or dual node CPU implementations.

  - **Ready to use MPESTI Expansion Capability**: Ready-to-deploy FPGA-based MPESTI fan-out solution with base BMC reference code



HPE Intel Lattice POC

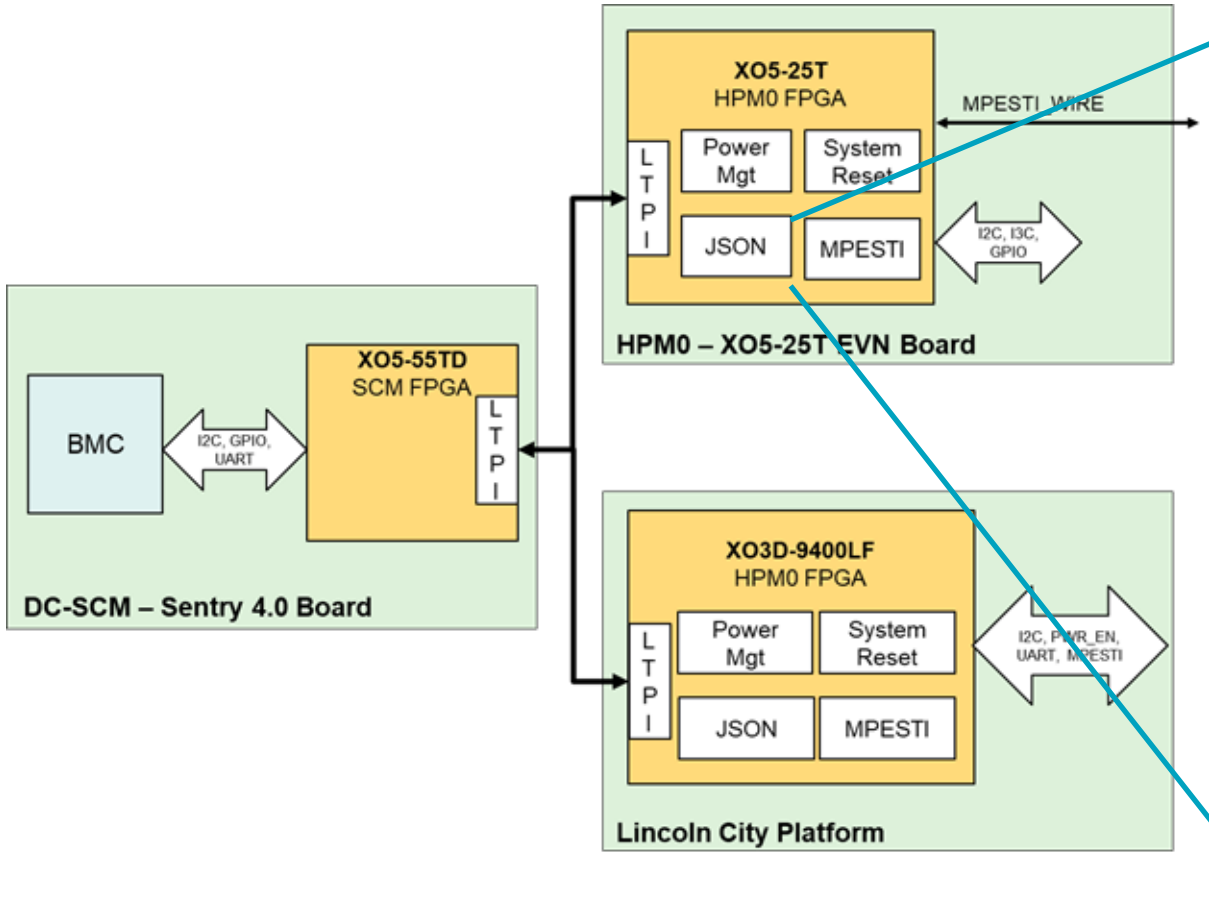
Standardizes BMC PnP firmware and FPGA capabilities for automatic discovery and integration of HPM platforms via HPM FPGA-JSON

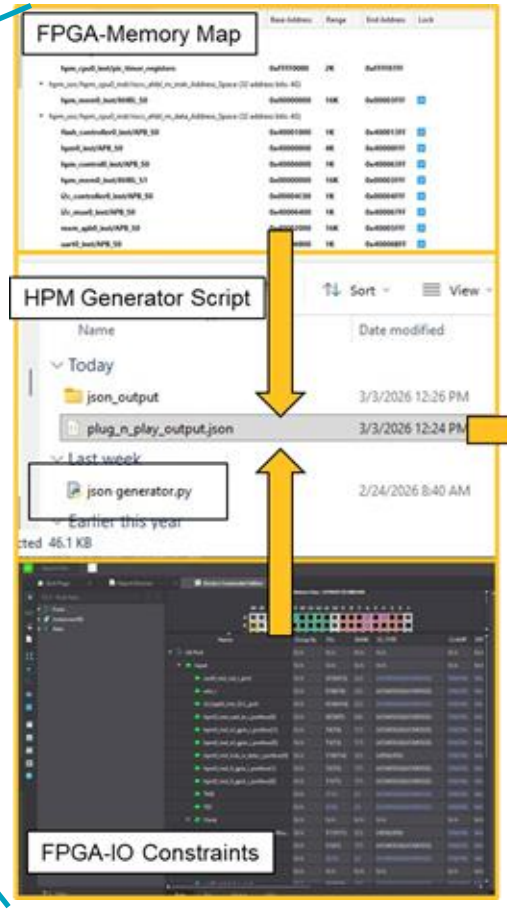


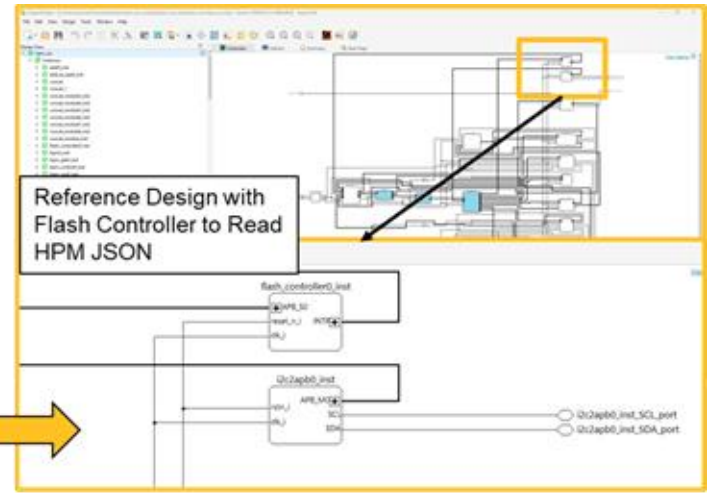
Design Template based on OCP HPM FPGA DESIGN 0.5 RC

HPE Intel Lattice POC

Automated JSON generation covers HPM FPGA, power management, and system reset workflows







- **Automatic generation of HPM JSON**
- **JSON saved on embedded Flash (reduce BOM for FRU)**
- **Reference Design to read JSON through I2C**

Reference Documents and Code Contributions

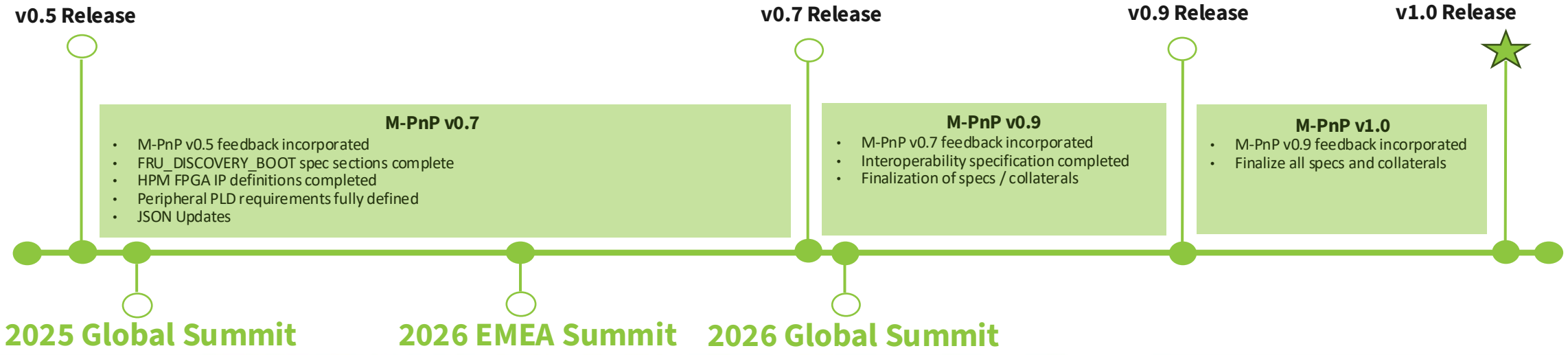
- HPE contributions
  - White paper for joint POC (currently in progress)
  - Tools: FRU Packaging Tool, Prog. Dev. JSON Generator, Netlist to JSON Generator, etc.
- Lattice contributions (currently in progress)
  - Provide a reference design with PnP IPs (PnP-Power Management, System Reset, Virtual Wire and sample firmware)
  - HPM Prog. Dev. (e.g., FPGA) JSON Generation tool (currently in progress)
- Dell Contributions
  - M-PESTI Verilog - https://github.com/opencomputeproject/OCP-SVR-MHS-PESTI
- Intel Contribution
  - LTPI Verilog - https://github.com/opencomputeproject/HWMgmt-Module-DCSCM-LTPI

M-PNP Industry Adoption Checklist

- [x] **Review M-PNP Specifications and JSON Mockups / Schemas**
  - Provide feedback on 0.7 specifications and JSON files
- [x] **Create Self-Describing MHS Component JSON, Device Tree, and FRU Files**
  - Experiment with the open-source tools and provide feedback
- [x] **Adopt M-PNP HW Support**
  - Engage R&D teams and plan towards building into future platforms

M-PNP Workstream Progress and Schedule

- **M-PnP v0.5 Specs released prior to OCP Global Summit ’25**
  - M-PNP.FRU_DISCOVERY_BOOT, M-PNP.FPGA, and M-PNP.INTEROP specifications
  - HPM and Peripheral Programmable Device JSON schemas and example mockup files
  - FRU Packaging Tool (frupack.py) was contributed to OCP by HPE
- PD JSON file creation tool (fpga2json.py) to contribute to OCP by HPE targeted Q4 ‘26
- **M-PnP v0.7 Specs targeted Oct ’26 – Soliciting Feedback now via webinars, outreach, etc.**
- Prototype JSON files, HPM PD IP blocks, and packaging/retrieval of FRU information files Q4 ‘26
- **M-PnP v0.9 targeted after OCP Global Summit Q4 ‘26 with 1.0 in Q1 ‘27**



Call to Action

Help us get to Plug-and-Play!

- Review the Tech Talks and collateral covering M-PNP specifications topics:
  - M-PNP Overview and FRU_Discovery_Boot Tech Talk: https://www.youtube.com/watch?v=QapIKDsCq3E
  - M-PNP FPGA Tech Talk: https://www.youtube.com/watch?v=sWQxhUJzc3k
  - Specs: https://www.opencompute.org/w/index.php?title=Server/MHS/DC-MHS-Specs-and-Designs#Latest_DC-MHS_Specifications
- Provide feedback through DC-MHS Project or get directly involved in the WS
  - Bring-in ideas & contribute to the design & interoperability specifications
- Help us prototype – use the open-source tools to create your own files (coming soon)
  - FRU Packaging Tool: https://github.com/opencomputeproject/OCP-SVR-MHS-M-PNP_FRUTool
- Stop by the M-PNP Innovation Station and booths to see demos and ask questions

Reach out to us through:

- DC-MHS Project Wiki: https://www.opencompute.org/wiki/Server/DC-MHS
- DC-MHS Public Mailing List: ocp-mhs-public@OCP-All.groups.io

Thank You!

